Generative Extensions for Reflection

Document #:	P3157R1
Date:	2024-05-22
Project:	Programming Language C++
Audience:	EWG, SG7

Reply-to: Andrei Alexandrescu, NVIDIA <andrei@nvidia.com>
Barry Revzin <barry.revzin@gmail.com>
Bryce Lelbach, NVIDIA <blelbach@nvidia.com>
Michael Garland, NVIDIA <garland@nvidia.com>

Contents

§ 1 Introduction

Since the recent implementation of the reflection facilities proposed in [P2996R3], we explored how the proposal would help fundamental challenges we are facing today. We believe reflection has great potential to solve important problems that we and the C++ community at large both face, and that a few specific enhancements on top of P2996 would help realize that potential. Based on our initial experience, this document (and its companions [P3294R0] and [P3289R0]) builds function synthesis capability on the foundation of P2996 that we think will help C++ reflection have a stronger and more timely positive impact on the state of affairs in the C++ language. We anticipate that C++ reflection, if sufficiently

powerful, could dramatically reduce costs of developing and maintaining C++ code while at the same time improving code size, readability, compilation time, and execution speed. We expect to have a proof-of-concept implementation of our design available shortly.

C++ code from a variety of domains naturally leads to boilerplate. Proxy classes are commonplace, whether to interface different codebases or as an organic part of design. The guideline "prefer composition over inheritance" leads to many forward-to-member functions. Other useful design patterns such as Visitor, Observer, Decorator, Adapter, and Null Object require de facto maintenance of parallel class hierarchies and/or parallel function declarations (plus in many cases mechanical definitions). Foreign language/API interfaces typically consist of large swaths of code following repetitive patterns. Implementing high-performance parallel algorithms typically requires specialized patterns for a variety of size, stride, and type combinations. All of these instances feature enough irregularities and variations to make existing template metaprogramming techniques difficult to deploy, and if deployed, difficult to understand and maintain.

The rest of this paper is structured as follows. Section "Function Descriptor Metafunctions" proposes a design for manipulating the reflection of functions, with an emphasis on generation capabilities. These capabilities—in conjunction with P2996, P3294, P3289, and P3096—provide a mechanism for querying and synthesizing function definitions in a powerful and flexible manner that makes reflection-based code simple and intuitive. Section "Proxy Classes and Instrumented Classes" defines essential use cases of reflection metaprogramming that motivate and inform the design and implementation of function synthesis. We consider strong use cases essential to setting goals for reflective metaprogramming. To be compelling, use cases must demonstrate meaningful, desirable functionality (albeit in a simplistic, proof-of-concept style) that is impossible or prohibitively difficult within the current C++. Section "Embedded Domain Specific Languages" is even more forward-looking, setting up the long-range trajectory of our proposal.

§ 2 Function Descriptor Metafunctions

This proposal is a companion to P3294 and meant to complement and work in conjunction with it. P3294 uses `decl_of(info)` (where `info` is the reflection of a function) as a key mechanism to expand reflections of existing functions into declarations, to which user code can subsequently attach definitions. This proposal focuses on creating and manipulating reflections of functions, to be later used with `decl_of(info)` as per P3294.

Given an existing function, we propose a number of *function descriptor metafunctions* that allow querying all aspects of it. Similar functions can be used to synthesize new functions.

Given a function declaration `f`, its reflection `^f` is a mutable object that can be subsequently modified. That does not affect the initial declaration in any way, but can be used to generate new declarations and definitions. Using `^f` once again returns a new copy of its reflection, so there is no loss of information.

The definitions of the proposed metafunctions are shown below. A subsection will be dedicated to describing each.

```
namespace std::meta {
  // inject a function definition given its reflection and body
  consteval auto inject_function(info, info) -> void;
   // inject a function definition given its reflection, parameter
        prefix, and body
  consteval auto inject_function(info, string_view, info) -> void;
   // create a new function declaration from tokens and return its
        reflection
  consteval auto declare_function(info) -> info;
  // present in P2996, applies to functions as well
  consteval auto name_of(info) -> string_view;
  // change the name of the function
  consteval auto set_name(info, string_view) -> void;
  // present in P2996, applies to functions as well
  consteval auto qualified_name_of(info) -> string_view;
  // change the qualfied name of the function
  consteval auto set_qualified_name(info, string_view) -> void;
  // get and set `static`
  consteval auto is_static(info) -> bool;
  consteval auto set_static(info, bool) -> void;
  // get and set `static`
  consteval auto is_virtual(info) -> bool;
  consteval auto set_virtual(info, bool) -> void;
  // get and set `static`
  consteval auto is_override(info) -> bool;
  consteval auto set_override(info, bool) -> void;
  // get and set `static`
  consteval auto is_final(info) -> bool;
  consteval auto set_final(info, bool) -> void;
  // "", "public", "protected", "private", or "friend"
  consteval auto access(info) -> string_view;
  consteval auto set_access(info, string_view) -> void;
  // "", "&", or "&&"
  consteval auto ref_qualifier(info) -> string_view;
  consteval auto set_ref_qualifier(info, string_view) -> void;
  // all user-defined attributes
  consteval auto attributes(info) -> vector<string>;
  consteval auto add_attribute(info, string_view) -> void;
  consteval auto remove_attribute(info, string_view) -> void;
  // true if the function is consteval
  consteval auto is_consteval(info) -> bool;
  consteval auto set_consteval(info, bool) -> void;
  // true if the function is constexpr
  consteval auto is_constexpr(info) -> bool;
  consteval auto set_constexpr(info, bool) -> void;
  // true if explicit ctor or conversion operator
  consteval auto is_explicit(info) -> bool;
  consteval auto set_explicit(info, bool) -> void;
  // true if the function is inline
  consteval auto is_inline(info) -> bool;
  consteval auto set_inline(info, bool) -> void;
  // indicates whether the function is pure virtual
  consteval auto is_pure(info) -> bool;
  consteval auto set_pure(info, bool) -> void;
  // indicates whether the function is deleted
```

```
  consteval auto is_deleted(info) -> bool;
  consteval auto set_deleted(info) -> void;
  // indicates whether the function is a default constructor or operator
  consteval auto is_default(info) -> bool;
  consteval auto set_default(info, bool) -> void;
}
```

§ 2.1 `inject_function`

These metafunctions inject a new function definition given the reflection of a function declaration and tokens for the function body. The optional `string_view` parameter provides a prefix for function parameters; by default, parameters are `_0`, `_1`, `_2` etc. For example:

```
double fun(double, std::string);
consteval {  // see P3289R0, "Consteval Blocks"
  // for @tokens see P3294, "Code Injection with Token Sequences"
  inject_function(^fun, @tokens{ return _0 + _1.size(); })
}
// Equivalent hand-written definition:
// double fun(double _1, std::string _2) { return _1 + _2.size(); }
```

If a prefix is specified, it will replace the leading underscore in parameter names. For example:

```
double fun(double, std::string);
consteval {
  inject_function(^fun, "p", @tokens{ return p0 + p1.size(); })
}
```

§ 2.2 `declare_function`

This metafunction creates a new function declaration from tokens and returns the reflection of that declaration. For example:

```
consteval {  // see P3289R0, "Consteval Blocks"
  auto r = declare_function(@tokens{ void my_function(double); });
  // see P3294, "Code Injection with Token Sequences"
  inject_function(r, @tokens{ std::cout << "Hello, world!\n"; });
}
```

The code above simply defines (albeit in an alembicated manner) the following function:

```
void my_function(double _1) {
  std::cout << "Hello, world!\n";
}
```

The advantage of using reflection is, of course, when there's a need to manipulate some elements of the function's definition prior to injection.

§ 2.3 `set_name`

This metafunction allows setting the name of a function's reflection, allowing user code to generate functions identical or similar in signature with existing functions. Example:

```
int f() noexcept;
consteval {
  auto r = ^f;
  set_name(r, "g");
  inject_function(r, @tokens{ return f(); });
}
// Equivalent to:
// int g() noexcept { return f(); }
```

Note how the signature including the `noexcept` attribute gets carried from `f` to `g`.

§ 2.4 `set_qualified_name`

Works similarly to `set_name`, but accepts a fully qualified name, which allows creating declarations and definitions outside the current namespace. This works per the existing language rules, i.e. the current scope must enclose the scope in which the function is injcted.

§ 2.5 `access`, `set_access`

The `access` metafunction returns the access level of the given reflection as one of the empty `std::string_view`, `"public"`, `"protected"`, `"private"`, or `"friend"`. If not applicable (e.g. top-level declaration), the empty `string_view` is returned. The setter metafunction `set_access` must be called with a `string_view` in the same set and forces the access level for the given reflection object.

§ 2.6 `ref_qualifier`, `set_ref_qualifier`

The `ref_qualifier` metafunction returns the access level of the given reflection as one of the empty `std::string_view`, `"&"`, or `"&&"`. If not applicable (e.g. top-level declaration), the empty `string_view` is returned. The setter metafunction `set_ref_qualifier` must be called with a `string_view` in the same set and forces the access level for the given reflection object.

§ 2.7 `is_static`, `set_static`, `is_virtual`, `set_virtual`, `is_override`, `set_override`, `is_final`, `set_final`, `is_consteval`, `set_consteval`, `is_constexpr`, `set_constexpr`, `is_explicit`, `set_explicit`, `is_inline`, `set_inline`, `is_pure` `set_pure`, `is_deleted`, `set_deleted`, `is_default`, `set_default`

These get/set metafunctions retrieve and set the respective aspects of the given reflection. Example:

```
static int f();
consteval {
  auto r = ^f;
  set_name(r, "g");
  static_assert(is_static(r));
  set_static(r, false);
  inject_function(r, @tokens{ return f(); });
}
// Equivalent to:
// int g() { return f(); }
```

Calling `is_xxx` on reflection objects where they are not applicable (e.g. reflections of data or expressions) returns `false`. Calling `set_xxx` where it isn't applicable results in a compile-time error.

§ 3 Proxy Classes and Instrumented Classes

Consider the task of defining interface classes for API integration, including foreign language bindings. Such a task involves creating classes with member functions that perform the same basic duties—such as validating arguments, converting data formats, and adjusting reference and pointer notations—before calling similarly named functions in another class, sometimes followed by analogous postprocessing of the results.

Such use cases generalize to creating *instrumented classes:* developing direct substitutes for existing classes while embedding specific hooks (such as tracing, logging, counters, argument verification, result validation, and naming convention changes) into some or all member functions. Successfully reflecting on and reconstructing a class in this manner serves as a critical test of a language's reflective metaprogramming capabilities, similar to how the identity function evaluates a language's functional programming features.

As a simple example, consider the Null Object design pattern, a safe alternative to the dreaded null pointer. Given an interface `T` that defines several pure virtual member functions, `null_object<T>` would yield an implementation of `T` that defines all of its virtuals to either throw an exception or return a default-constructed value of their result type. Defining a null object for a given interface is tediously simple, yet maintaining it is an exercise in frustration and a source of aggravation. Reflection should allow defining `null_object` for any type and behavior with ease.

A more involved example would be defining a class such as `instrumented_vector<T, A>` that wraps an `std::vector<T, A>` and adds instrumentation (e.g., bounds checking) to some or all of its methods. The key here is to make `instrumented_vector` a drop-in replacement for `std::vector` without incurring the cost of copying all of its member function declarations. Needless to say, defining such a proxy class by hand is quite discouraging, and reflective metaprogramming should offer a complete and flexible solution.

The ability to create proxy classes would also put to rest the unpleasantness of following the adage "prefer composition over inheritance" in C++. As mentioned, in many composition situations, numerous forward-to-member functions must be written and maintained; automating these stubs would make it much easier to follow the guideline therefore improving code quality without adding to its bulk. Making valuable programming idioms more accessible has repeatedly proven to be a wise investment.

An important part of defining instrumented classes is querying all members of an existing class (static and nonstatic data member, regular and special member functions, `enum` declarations, friend declarations…) and generating similar definitions within the context of a new class definition. The `define_class` primitive in P2996R1 is the fundamental mechanism for implementing a proxy class, and although it currently does not support adding member functions, it alludes to such a possibility in section 4.4.12: "For now, only non-static data member reflections are supported (via `nsdm_description`) but the API takes in a range of `info` anticipating expanding this in the near future." We believe the ability to define full-fledged classes is a quintessential, defining feature of a reflective metaprogramming feature for C++. Here are a few key components needed:

— Signatures of all functions must be accessible for introspection, and primitives for accessing full information of a function's signature must be defined.

— Synthesis of function signatures must be possible, e.g. a library may need to build a signature from scratch, or from a similar signature (e.g., create a new signature from a given signature by adding or removing an attribute).

— There must be an ability to attach code to the reflection of a function signature; for example, a library may want to define a proxy class that inserts logging for each function's arguments and result. The most fit candidate for attaching such functionality to reflection is a generic function literal that is a friend of the generated class.

— Finally, `define_class` would accept synthesized member functions in addition to (and in a manner similar to) `nsdm_description`. Implementing member function synthesis should be feasible following a design similar to `nsdm_description`—a function `memfun_description` would take an `std::meta::info` (either synthesized or coming from the introspection of another member function) a `memfun_options` object that has, among other members, a lambda function to serve as the body of the budding member function. The result of the call to `memfun_description` would be passed to `define_class`.

One aspect of function synthesis is *code cloning*—the ability to compile a reflected function template under different constraints (e.g. different concepts and attributes). As an example, this aspect is important to CUDA C++ libraries that need to add `__device__` attributes to all methods of replicas of standard types—such as `std::pair`, `std::tuple`, and `std::optional`—and subsequently compile the resulting code for use on the device. The alternative—copying and pasting code with minute changes—is a proverbially bad practice.

§ 4 Embedded Domain Specific Languages

Python's many frameworks (e.g., PyTorch and TensorFlow, among many others) have demonstrated the importance of Embedded Domain Specific Languages (EDSLs) in today's world, especially for AI

applications. A reflective metaprogramming facility for C++ is expected to make it possible to express EDSLs much better than C++ currently allows.

EDSL-related features would place emphasis on the *generative* aspect of reflective metaprogramming. For example, an aspirational EDSL way of doing things for GPU-accelerated code could take in an algorithm written concisely in a high-level array language and generate during compilation specialized CUDA C++ code implementing the algorithm with the same efficiency as if this low-level code were written by hand.

The utility of EDSLs is, of course, much broader. An EDSL essentially allows the programmer to author a high-level expressive specification adapted to the problem domain (function differentiation, relational databases, networking protocols, regular expressions, EBNF grammars, document formatting…), to then generate C++ code from it. The approach is advantageous if writing the same C++ code by hand would be a much more costly proposition. Domain-specific libraries can provide the desired level of abstraction, but struggle to optimize execution in ways that cross abstraction boundaries. EDSL-style approaches can provide this missing capability.

The *amplification* aspect is an important desired outcome: the ultimate goal of a reflection facility is to allow new code to build on existing code in a combinatorial manner, to the effect of automating tedious and repetitive aspects of the coding process. To wit, the examples in P2996R1 and those shown above invariably would take more code to implement without reflection. Allowing generation of meaningful code from specifications encoded in EDSLs would be the ultimate goal of a reflection engine.

EDSL processors (toolchains that act as embedded interpreters or compilers) play a pivotal role in the EDSL ecosystem. Though an EDSL processor itself can be seen as an ordinary EDSL that takes in a grammar specification and produces a language translator, once the EDSL toolchain is available it can be used to process any other EDSL, either during compilation or at runtime. Therefore, an EDSL toolchain defined as an EDSL is a foundational "seed EDSL" that we consider a crucial milestone of a reflection metaprogramming engine.

For such a task to be feasible, the *synthesis* aspect of the proposal is important—building on the `memfun_description` function discussed above, we envision adding primitives that synthesize types, functions, and templates in addition to the `define_class`/`nsdm_description` facility that allows definition of classes with direct data members.

§ 5 References

[P2996R3] Barry Revzin, Wyatt Childers, Peter Dimov, Andrew Sutton, Faisal Vali, Daveed Vandevoorde, and Dan Katz. 2024-05-22. Reflection for C++26.
https://wg21.link/p2996r3

[P3289R0] Wyatt Childers, Barry Revzin, and Daveed Vandevoorde. 2024-05-18. `consteval` blocks.
https://wg21.link/p3289r0

[P3294R0] Andrei Alexandrescu, Barry Revzin, and Daveed Vandevoorde. 2024-05-22. Code Injection with Token Sequences.
https://wg21.link/p3294r0